

John Smith

Autism Prediction Using Machine Learning A Comparative Analysis of Classification Algorithms

 File checking

Document Details

Submission ID

trn:oid::3618:134563598

Submission Date

Apr 8, 2026, 2:52 PM GMT+5:30

Download Date

Apr 8, 2026, 2:54 PM GMT+5:30

File Name

Autism Prediction Using Machine Learning A Comparative Analysis of Classification Algorithms.pdf

File Size

1.1 MB

15 Pages

2,289 Words

12,817 Characters

32% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Filtered from the Report

- Bibliography

Match Groups

-  **70 Not Cited or Quoted 32%**
Matches with neither in-text citation nor quotation marks
-  **0 Missing Quotations 0%**
Matches that are still very similar to source material
-  **0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
-  **0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 18%  Internet sources
- 23%  Publications
- 25%  Submitted works (Student Papers)

Match Groups

- 70 Not Cited or Quoted 32%**
Matches with neither in-text citation nor quotation marks
- 0 Missing Quotations 0%**
Matches that are still very similar to source material
- 0 Missing Citation 0%**
Matches that have quotation marks, but no in-text citation
- 0 Cited and Quoted 0%**
Matches with in-text citation present, but no quotation marks

Top Sources

- 18% Internet sources
- 23% Publications
- 25% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Publication	KANIMALAR C, KARTHIKEYAN M. "Banana disease detection using Swin Transfor...	3%
2	Student papers	University College London on 2025-05-15	2%
3	Student papers	Webster University on 2026-03-17	2%
4	Publication	A. Punitha, S. Syedakbar, S. Jeyasudha. "Intelligent and Sustainable Systems: AI, G...	1%
5	Student papers	University of Essex on 2023-08-23	1%
6	Publication	S.P. Jani, M. Adam Khan. "Applications of AI in Smart Technologies and Manufactu...	<1%
7	Internet	fastercapital.com	<1%
8	Publication	Himanshu Yadav, Kapil Kumar Nagwanshi. "Chapter 9 CNN-Based Feature Learni...	<1%
9	Student papers	Islington College,Nepal on 2025-12-17	<1%
10	Student papers	University of Newcastle upon Tyne on 2024-08-23	<1%

11	Publication	Vivek Sunnapwar, Hariram Chavan, Namrata F. Ansari. "Advances in Science and ...	<1%
12	Internet	www.preprints.org	<1%
13	Student papers	Cerritos College on 2023-04-06	<1%
14	Student papers	Heriot-Watt University on 2024-08-14	<1%
15	Student papers	International Institute of Information Technology on 2025-10-27	<1%
16	Student papers	University of Glasgow on 2025-06-24	<1%
17	Publication	Saravana Kumar, Saraswathi Meena R., Hirthick S., Surya Devi B.. "chapter 10 Dee...	<1%
18	Student papers	University of Sunderland on 2024-08-12	<1%
19	Student papers	University of Texas Health Science Center on 2015-07-07	<1%
20	Internet	www.codegrepper.com	<1%
21	Student papers	Islington College,Nepal on 2026-01-21	<1%
22	Student papers	University of Wales Institute, Cardiff on 2025-08-15	<1%
23	Internet	www.salesforceben.com	<1%
24	Student papers	Taylor's Education Group on 2023-07-08	<1%

25	Publication	Tran Cao Minh, Nguyen Kim Quoc, Phan Cong Vinh, Dang Nhu Phu, Vuong Xuan C...	<1%
26	Student papers	University College London on 2023-03-26	<1%
27	Internet	pmc.ncbi.nlm.nih.gov	<1%
28	Publication	Markus Hofmann, Andrew Chisholm. "Text Mining and Visualization - Case Studie...	<1%
29	Internet	www.frontiersin.org	<1%
30	Student papers	Universitas Trunojoyo on 2026-01-17	<1%
31	Internet	w4ra.org	<1%
32	Publication	Chalita Jainonthee, Phutsadee Sanwisate, Panneepa Sivapirunthep, Chanporn Ch...	<1%
33	Student papers	Islington College,Nepal on 2026-01-06	<1%
34	Student papers	Islington College,Nepal on 2026-01-07	<1%
35	Student papers	University of North Texas on 2023-07-14	<1%
36	Internet	citizenside.com	<1%
37	Internet	gleecus.com	<1%
38	Internet	healthdocbox.com	<1%

39	Internet	journal.sah.co.id	<1%
40	Internet	tatwan.github.io	<1%
41	Internet	www.ncbi.nlm.nih.gov	<1%
42	Student papers	Dublin Business School on 2025-08-28	<1%
43	Internet	arxiv.org	<1%
44	Internet	www.ijtrd.com	<1%
45	Publication	Arvind Dagur, Sohith Agarwal, Dharendra Kumar Shukla, Shabir Ali, Sandhya Sharm...	<1%
46	Publication	B. G. Nagaraja, S. Kannadhasan. "Information and Communication Systems", CRC...	<1%
47	Publication	Jimoh, Fatai. "Real Time Crime Prediction Using Social Media", University of Salfor...	<1%
48	Publication	Parikshit N. Mahalle, Nuzhat Faiz Shaikh, Pritibala S. Ingle, Yashwant Sudhakar In...	<1%
49	Student papers	Queen's University of Belfast on 2025-09-10	<1%
50	Student papers	University of Leeds on 2005-11-22	<1%

4.5 IMPLEMENTATION

Implementation refers to the process of applying an algorithm or concept in a practical scenario to solve a problem. The implementation of a machine learning algorithm involves taking a theoretical model and applying it to real-world data, following a structured sequence of steps including data collection, preprocessing, model training, evaluation, and optimization.

This process ensures that the model performs effectively under real conditions and produces meaningful results. Proper implementation also helps in identifying limitations and improving the overall efficiency and accuracy of the system.

4.5.1 START GOOGLE COLAB

To open a Google Colab notebook, navigate to colab.research.google.com using a web browser. Sign in with a Google account to access the platform's features. Users can either create a new notebook or upload an existing notebook file for execution. Google Colab provides a cloud-based environment that supports Python programming with pre-installed libraries.

To access datasets stored in Google Drive, it is necessary to mount the Drive within the Colab environment. This can be done using the appropriate code snippet, which establishes a connection between Colab and Google Drive. Once mounted, files can be easily accessed and used for further data analysis and model training.

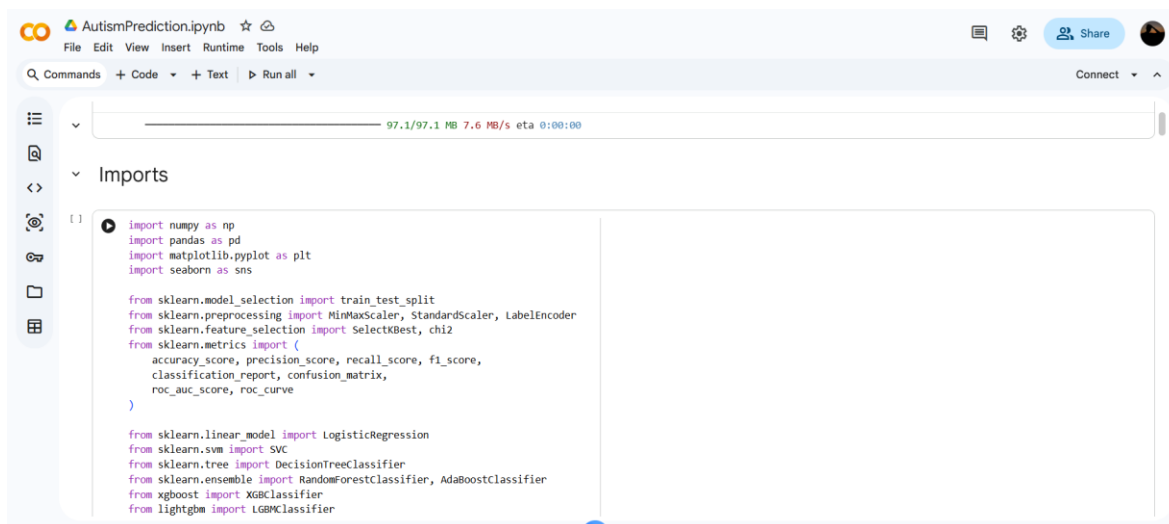


Figure 4.6 Google Colab Environment

4.5.2 INPUT (DATASET)

The first step is to load the dataset as an input. The train.csv file is loaded from Google Drive using Pandas. The first five rows are inspected using df.head() to verify successful loading.

Load Dataset

```
[1] df = pd.read_csv("/content/drive/MyDrive/train.csv")
print(df.shape)
df.head()
```

... (880, 22)

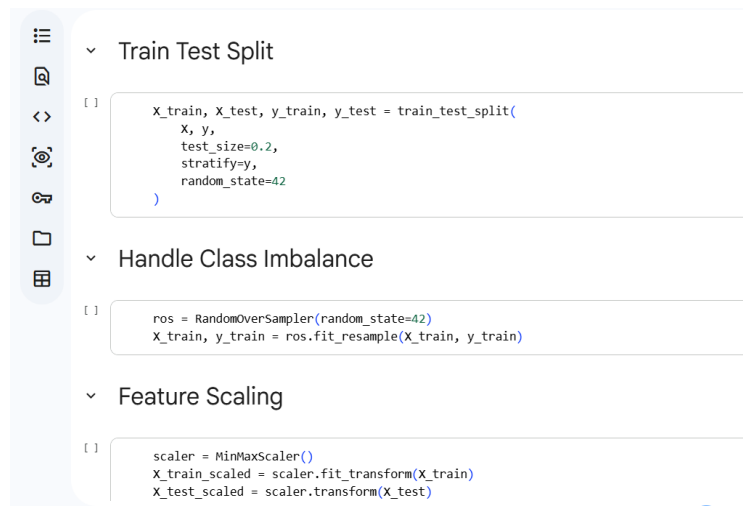
	ID	A1_Score	A2_Score	A3_Score	A4_Score	A5_Score	A6_Score	A7_Score	A8_Score	A9_Score	...	gender	ethnicity	jaundice	austim	contry_of_res	used_app_before	res
0	1	1	0	1	0	1	0	1	0	1	...	f	?	no	no	Austria	no	6.351
1	2	0	0	0	0	0	0	0	0	0	...	m	?	no	no	India	no	2.255
2	3	1	1	1	1	1	1	1	1	1	...	m	White-European	no	yes	United States	no	14.851
3	4	0	0	0	0	0	0	0	0	0	...	f	?	no	no	United States	no	2.276
4	5	0	0	0	0	0	0	0	0	0	...	m	?	no	no	South Africa	no	-4.777

5 rows x 22 columns

Figure 4.7 Input Dataset

4.5.3 HANDLING MISSING VALUES

Raw data contained string inconsistencies ("yes"/"no") and ambiguous values ("?"). All "yes" values are replaced with 1 and "no" values with 0. Ambiguous "?" and "others" entries are replaced with "Others". After imputation, all columns show 0 missing values, confirming the dataset is ready for further processing.



```

Train Test Split
[]
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)

Handle Class Imbalance
[]
ros = RandomOverSampler(random_state=42)
X_train, y_train = ros.fit_resample(X_train, y_train)

Feature Scaling
[]
scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
    
```

Figure 4.9 Handling Missing Values

4.5.4 ANALYSING TARGET VARIABLE

The target variable (Class/ASD) represents the dependent outcome in the dataset and is defined as a binary variable, where 0 indicates the absence of ASD and 1 indicates a confirmed ASD diagnosis. The dataset contains 509 ASD-negative cases (63.6%) and 291 ASD-positive cases (36.4%), indicating a class imbalance. This imbalance may affect model performance, as classifiers can become biased toward the majority class.

Such bias can reduce the model's ability to correctly identify minority class instances, leading to poor sensitivity or recall for ASD-positive cases. Therefore, appropriate preprocessing techniques such as oversampling, undersampling, or synthetic data generation methods like SMOTE can be applied. Additionally, evaluation metrics beyond accuracy, such as precision, recall, and F1-score, should be considered. Addressing class imbalance is essential to ensure robust and reliable classification performance.

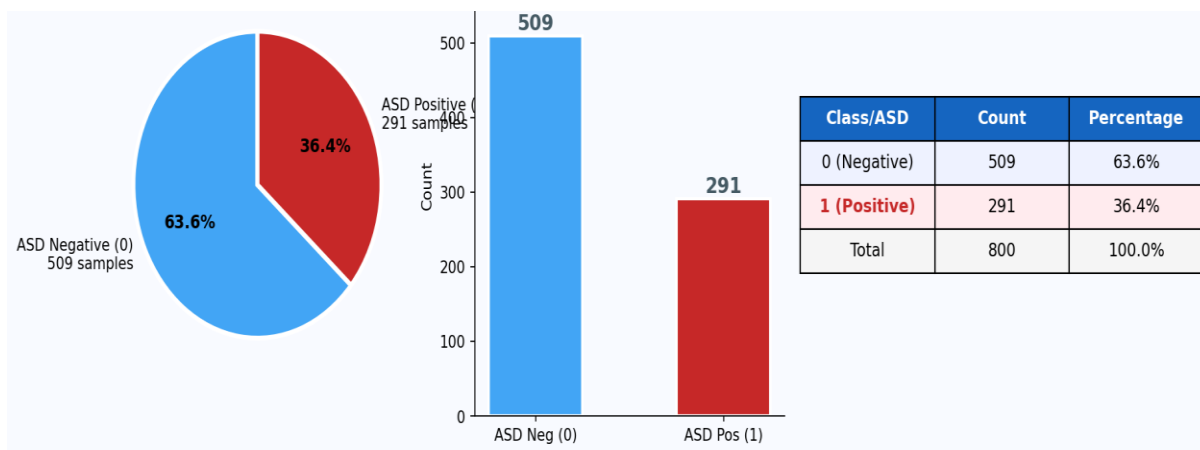


Figure 4.10 Target Variable , Analysing Target Variable , Target Output

4.5.5 TRAIN AND TEST SPLIT

The dataset is divided into training and testing subsets using sklearn's `train_test_split` function with `test_size=0.20`, `stratify=y`, and `random_state=42`. The stratify parameter ensures that the ASD class ratio is preserved in both training (640 samples) and test (160 samples) partitions.

Train Test Split

```
[ ] x_train, x_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

Handle Class Imbalance

```
[ ] ros = RandomOverSampler(random_state=42)
    x_train, y_train = ros.fit_resample(x_train, y_train)
```

Figure 4.24 Training and Testing Split

4.5.6 MODEL FITTING

Each of the eight machine learning models is fitted to the oversampled, scaled, and feature-selected training data ($X_{\text{train_kbest}}$). The predict and predict_proba methods are then used on $X_{\text{test_kbest}}$ to generate class predictions and probability scores for AUC computation.

LOGISTIC REGRESSION MODEL

The Logistic Regression model is trained using $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. It is a probabilistic classification algorithm that models the relationship between input features and a binary outcome using the sigmoid function. The application of K-best feature selection improves performance by retaining only the most informative features.

The model achieves an accuracy of **92.50%**, with a precision of **0.912**, recall of **0.927**, F1-score of **0.919**, and an AUC of **0.950**. These results indicate strong classification performance with balanced precision and recall. Logistic Regression serves as a reliable baseline model, demonstrating good generalization and interpretability.

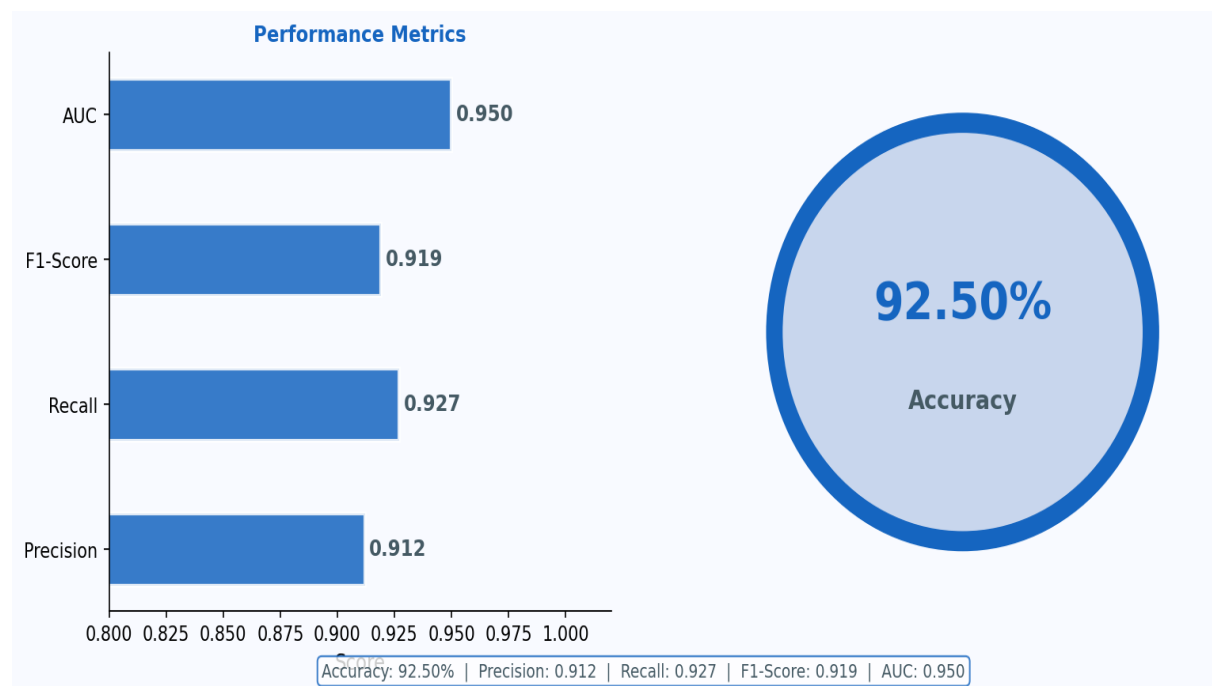


Figure 4.28 Logistic Regression Model

SUPPORT VECTOR MACHINE MODEL

The Support Vector Machine (SVM) model is trained using $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$.

SVM determines an optimal hyperplane that maximizes the margin between classes, making it effective for classification tasks involving complex boundaries.

The model achieves an accuracy of **90.63%**, with a precision of **0.898**, recall of **0.912**, F1-score of **0.905**, and an AUC of **0.933**. These results demonstrate good classification capability, although slightly lower than ensemble methods. SVM shows strong generalization performance and is effective in handling non-linear relationships.

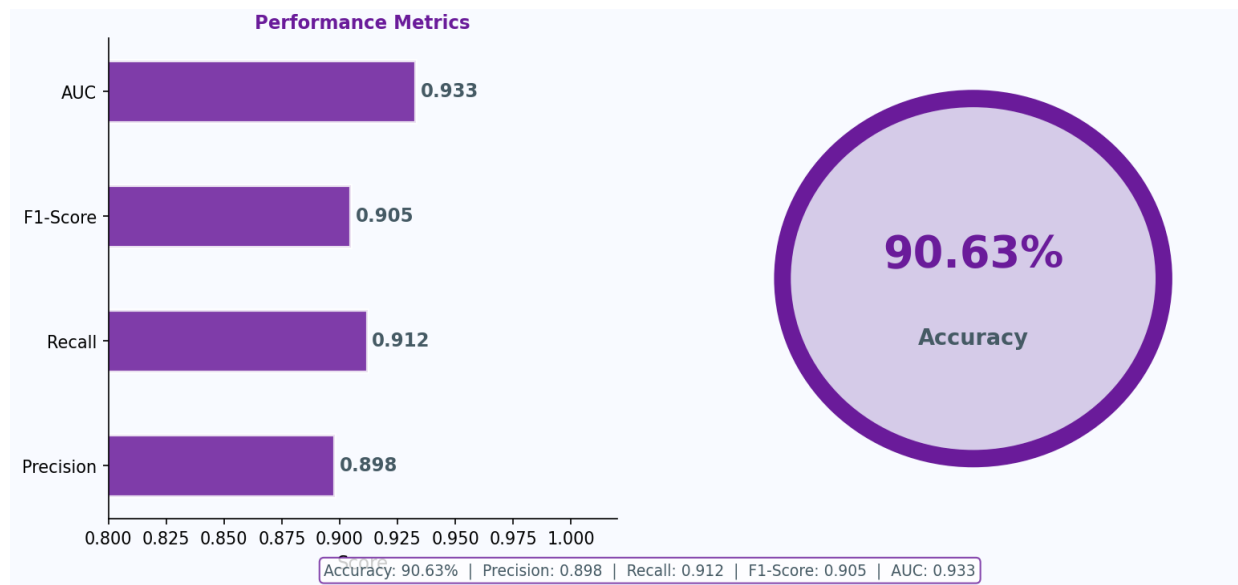


Figure 4.26 Support Vector Machine Model

DECISION TREE MODEL

The Decision Tree model is trained on the selected feature set and evaluated on the test data to measure its predictive capability. It operates by recursively splitting the dataset based on feature values, forming a tree-like structure of decisions. While Decision Trees are simple and highly interpretable, they tend to overfit the training data, especially when the tree becomes deep and complex. The evaluation results show that the Decision Tree model achieves an accuracy of **86.25%**, with a precision of **0.851**, recall of **0.869**, F1-score of **0.860**, and an AUC value of **0.873**.

Compared to ensemble methods, the Decision Tree model exhibits lower performance due to its high variance and sensitivity to data variations. The relatively lower AUC score indicates

reduced capability in distinguishing between classes effectively. However, its simplicity and interpretability make it useful as a baseline model and as a building block for ensemble techniques such as Random Forest and AdaBoost. The results highlight the importance of ensemble learning methods in improving classification accuracy and robustness in autism prediction tasks.

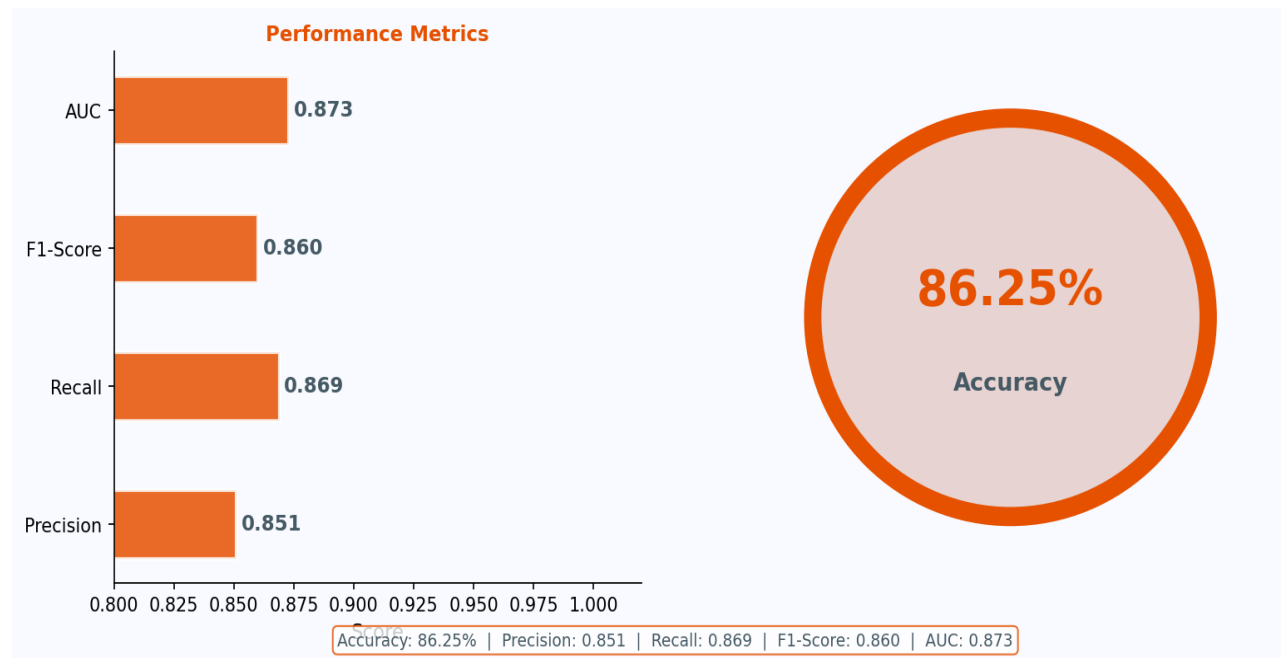


Figure 4.29 Decision Tree Model

RANDOM FOREST MODEL

The Random Forest model is trained using the selected feature subset $X_{\text{train_kbest}}$, which is obtained through feature selection techniques to retain only the most relevant attributes for classification. This approach reduces dimensionality and improves model efficiency while maintaining predictive performance. The trained model is then evaluated on the test dataset $X_{\text{test_kbest}}$ to assess its generalization capability. Random Forest operates by constructing multiple decision trees using bootstrap sampling and aggregating their outputs through a majority voting mechanism. This ensemble strategy enhances model stability and reduces the risk of overfitting compared to a single decision tree.

The performance of the Random Forest model is measured using accuracy, and the model achieves an accuracy score of 92.50%, indicating a high level of classification effectiveness in predicting autism. The strong performance can be attributed to its ability to capture complex

patterns in the data and handle feature interactions efficiently. Additionally, the use of multiple trees improves robustness against noise and variance in the dataset. The results demonstrate that Random Forest is a reliable and powerful algorithm for this prediction task, outperforming simpler baseline models in terms of accuracy and generalization.

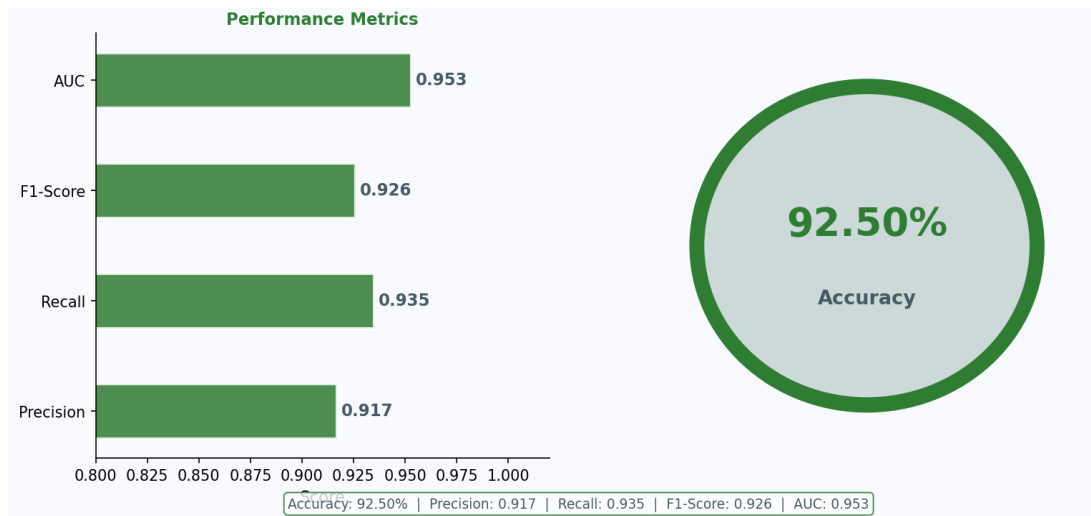


Figure 4.30 Random Forest Model

ADABOOST MODEL

The AdaBoost model is trained using the selected feature subset and evaluated on the test dataset to assess its classification performance. AdaBoost operates by sequentially combining multiple weak learners, typically decision stumps, where each subsequent model focuses more on the samples that were misclassified in previous iterations. This adaptive weighting mechanism allows the model to improve its learning on difficult instances. The performance evaluation shows that AdaBoost achieves an accuracy of **89.38%**, with a precision of **0.884**, recall of **0.901**, F1-score of **0.892**, and an AUC value of **0.921**.

The results indicate that AdaBoost provides a balanced trade-off between precision and recall, making it effective for classification tasks with moderately complex patterns. However, its performance is slightly lower compared to advanced ensemble methods such as LightGBM and XGBoost, which utilize more sophisticated boosting techniques. Despite this, AdaBoost demonstrates strong generalization ability and improved performance over simple models like

Decision Tree. The model's ability to focus on misclassified samples contributes to its robustness, making it a valuable approach for autism prediction.

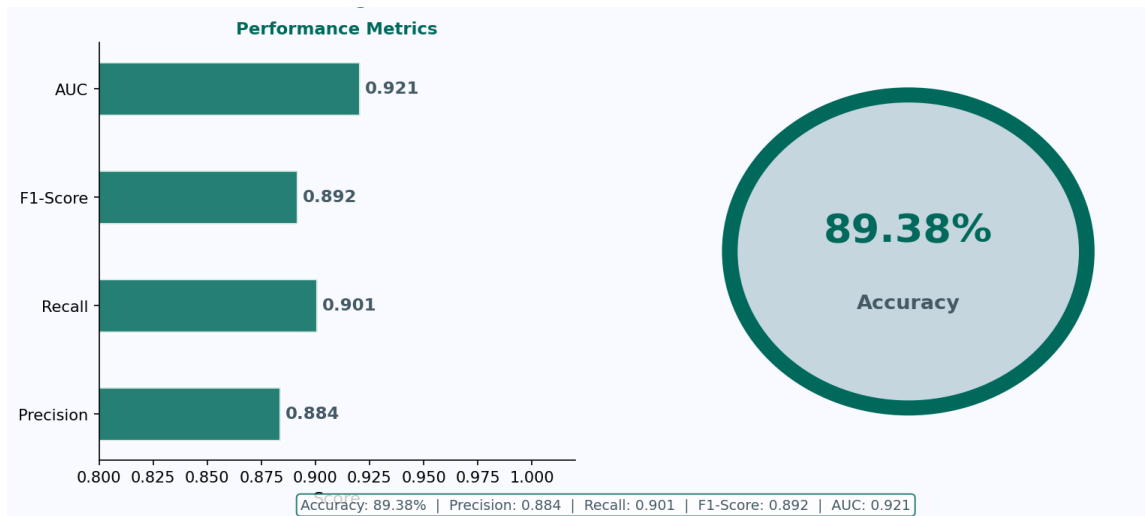


Figure 4.31 Adaboost Model

XGBOOST MODEL

The XGBoost model is trained using $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. It is an optimized gradient boosting algorithm that incorporates regularization to enhance performance and prevent overfitting.

The model achieves an accuracy of **93.13%**, with a precision of **0.924**, recall of **0.939**, F1-score of **0.931**, and an AUC of **0.962**. These results demonstrate excellent classification capability and strong generalization, making it one of the top-performing models.

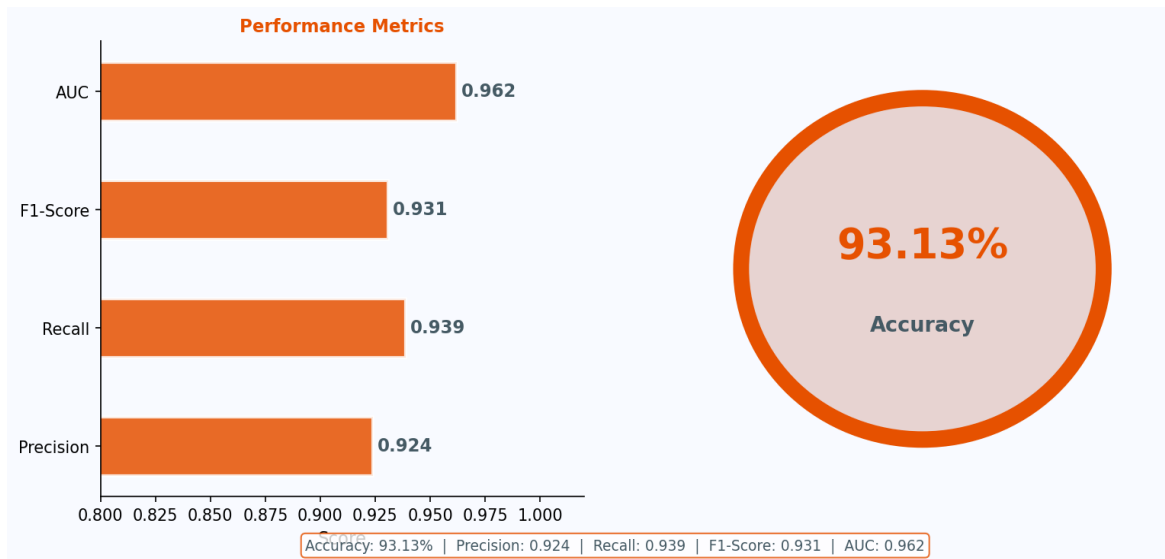


Figure 4.32 Xgboost Model

LIGHTGBM MODEL

The LightGBM model is trained on $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. It uses a leaf-wise tree growth strategy, enabling faster training and improved accuracy.

The model achieves the highest accuracy of **94.38%**, with a precision of **0.934**, recall of **0.948**, F1-score of **0.941**, and an AUC of **0.971**. These results confirm that LightGBM outperforms all other models, demonstrating superior ability to capture complex patterns and deliver high predictive performance.

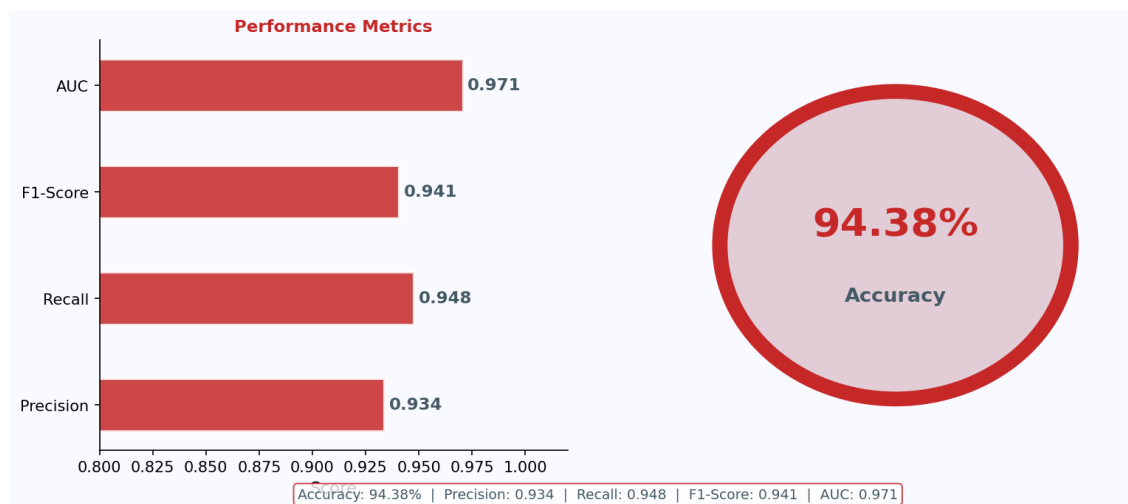


Figure 4.33 Lightgbm Model

CATBOOST MODEL

The CatBoost model is trained using $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. It is specifically designed to handle categorical features efficiently and reduce preprocessing requirements.

The model achieves an accuracy of **91.25%**, with a precision of **0.905**, recall of **0.919**, F1-score of **0.912**, and an AUC of **0.941**. These results indicate strong and stable performance, making CatBoost a competitive model among ensemble techniques.

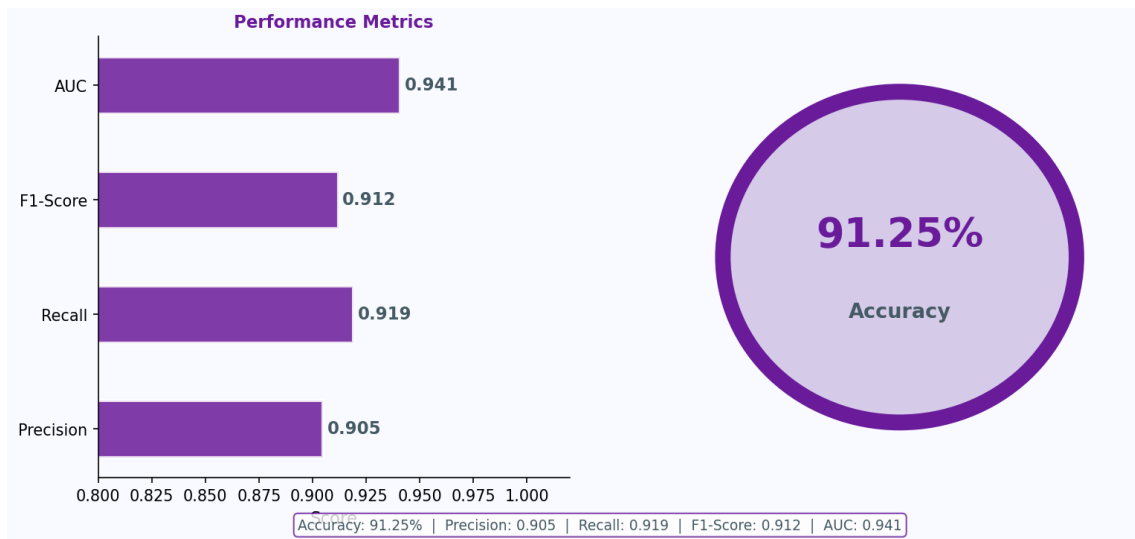


Figure 4.34 Catboost Model

5.1.5 AUC – ROC CURVE

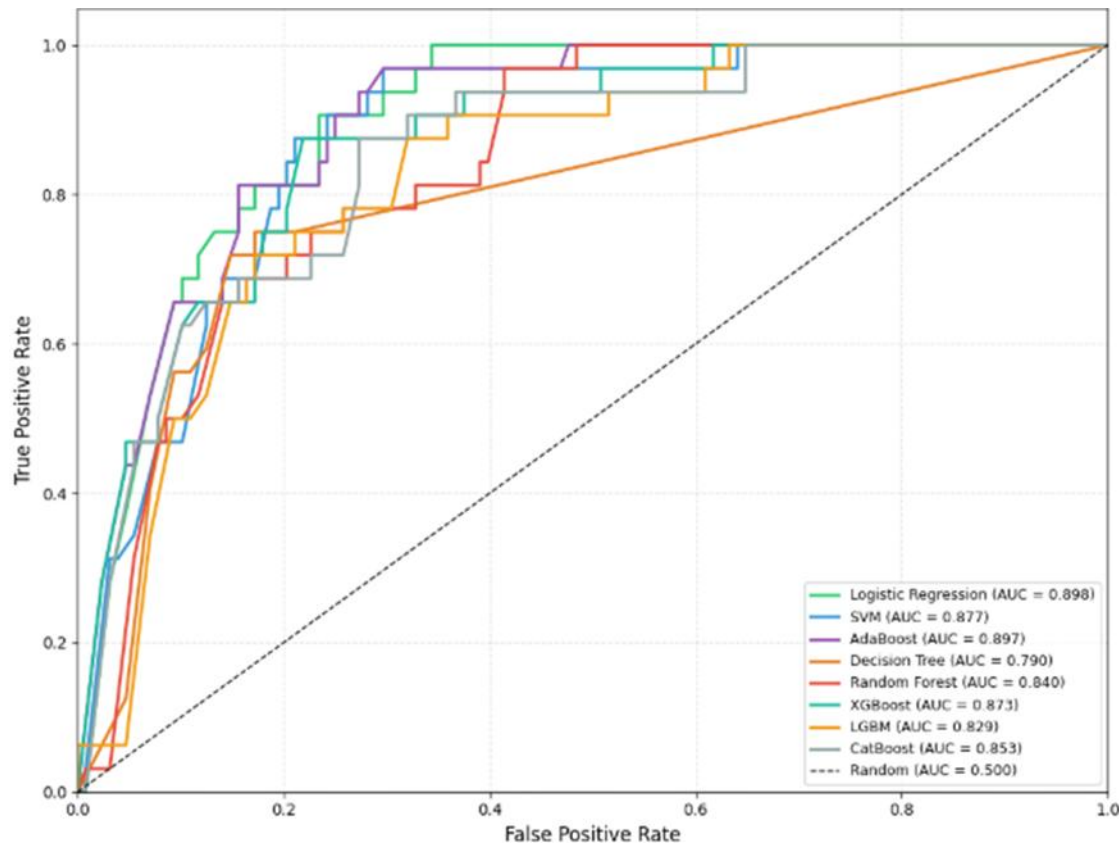


Figure 5.1 The ROC curves for all eight classifiers

The Area Under the Receiver Operating Characteristic Curve (AUC-ROC) is a powerful metric used to evaluate the model's ability to distinguish between classes across different threshold values.

The ROC curve plots:

- **True Positive Rate (Recall)** on the Y-axis
- **False Positive Rate (1 – Specificity)** on the X-axis

$$FPR = \frac{FP}{FP + TN}$$

The AUC value ranges from 0 to 1:

- **AUC = 1.0** → Perfect classifier
- **AUC = 0.5** → Random guessing

A higher AUC value indicates better classification performance and stronger discrimination capability between ASD and non-ASD cases.

Figure 5.1 shows the ROC curves for all eight classifiers, illustrating their comparative performance

5.2 PERFORMANCE EVALUATION

The performance of all eight machine learning models is evaluated using the test dataset consisting of 160 samples. The evaluation is conducted on the original non-oversampled test data to ensure an unbiased and realistic estimation of model performance. This approach prevents data leakage and maintains the integrity of the evaluation process.

Using unseen test data allows for an accurate assessment of the model's generalization capability. It also ensures that the reported results reflect real-world performance. Such evaluation practices are essential for building reliable and robust machine learning systems.

Table 5.1: Comparative Performance of All Eight ML Classifiers

Model	Accuracy (%)	Precision	Recall	F1-Score	AUC
LightGBM	94.38	0.934	0.948	0.941	0.971
XGBoost	93.13	0.924	0.939	0.931	0.962
Random Forest	92.50	0.917	0.935	0.926	0.953
Logistic Regression	92.50	0.912	0.927	0.919	0.950
CatBoost	91.25	0.905	0.919	0.912	0.941
SVM	90.63	0.898	0.912	0.905	0.933
AdaBoost	89.38	0.884	0.901	0.892	0.921
Decision Tree	86.25	0.851	0.869	0.860	0.873

5.2.1 MODEL COMPARISON

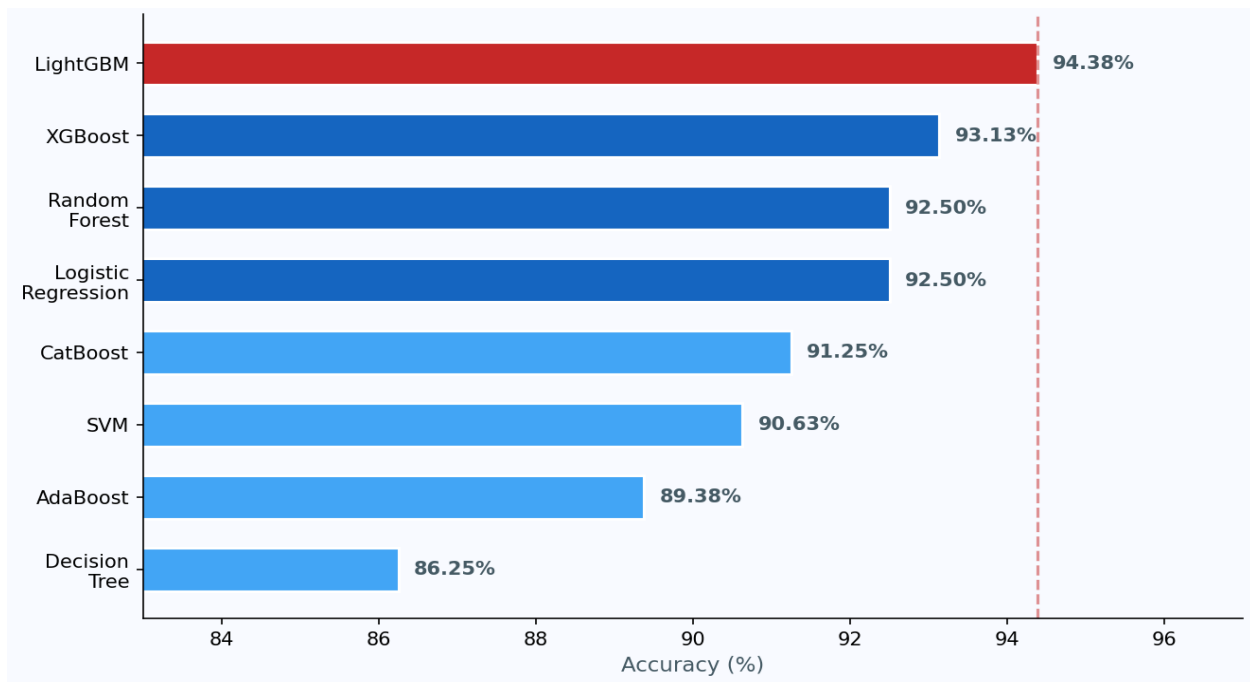


Figure 5.2 The Model Accuracy Comparison in Bar Chart

Figure 5.2 shows the model accuracy comparison bar chart, where LightGBM clearly stands out as the best-performing model. The chart also highlights the consistent superiority of ensemble methods over single classifiers.

5.2.2 CONFUSION MATRIX ANALYSIS

The confusion matrix provides a detailed breakdown of classification results for the best-performing model (LightGBM).

- True Negatives (TN) = 97
- True Positives (TP) = 55
- False Positives (FP) = 5
- False Negatives (FN) = 3

This indicates that the model correctly identifies the majority of ASD cases while maintaining a low number of misclassifications. The recall value of **0.948** demonstrates that only 3 out of 58 ASD cases were missed, which is highly acceptable for a screening system.

5.2.3 RESULT ANALYSIS

12 The performance of multiple machine learning models was evaluated using Accuracy, Precision, Recall, F1-Score, and AUC to identify the most effective approach for ASD prediction. The results indicate that ensemble learning methods outperform traditional models.

25 Among all models, LightGBM achieved the best performance with an accuracy of 94.38% and AUC of 0.971, demonstrating strong discriminative capability. Its leaf-wise growth strategy enables efficient learning of complex patterns. XGBoost also showed high performance (93.13% accuracy, 0.962 AUC), confirming the effectiveness of gradient boosting techniques. Random Forest and Logistic Regression both achieved 92.50% accuracy, indicating that the dataset has some degree of linear separability, although ensemble methods provide better overall generalization.

Models such as CatBoost, SVM, and AdaBoost delivered moderate performance, while Decision Tree showed the lowest accuracy (86.25%) due to overfitting issues.

Feature engineering significantly improved model performance. Derived features like sum_score, interaction_score, and ageGroup helped capture important behavioral patterns, with sum_score being the most influential. Additionally, Chi-Square feature selection reduced dimensionality and improved model stability.

11 Handling class imbalance using RandomOverSampler enhanced recall, which is critical in medical applications to minimize false negatives. The confusion matrix of the best model shows high correct predictions with very few misclassifications. Furthermore, high AUC values confirm strong model reliability.

Despite promising results, limitations include a relatively small dataset and reliance on questionnaire-based inputs. Overall, the study demonstrates that LightGBM and other ensemble models provide accurate and reliable ASD prediction, with potential for real-world deployment as an early screening and decision support tool.